

ASSIGN6

September 16, 2026

```
[1]: import numpy as np

print("=" * 70)
print("UCS420: COGNITIVE COMPUTING - ASSIGNMENT 6")
print("NUMPY-II")
print("=" * 70)
```

```
=====
UCS420: COGNITIVE COMPUTING - ASSIGNMENT 6
NUMPY-II
=====
```

```
[2]: # Q1. SENSOR TEMPERATURE READINGS
# =====

print("\n\n" + "=" * 70)
print("Q1. SENSOR TEMPERATURE READINGS")
print("=" * 70)

temperature = np.array([25, 28, 31, 35, 38, 27, 33, 40])

# a. Add 2°C to every reading using vectorization
corrected_temperature = temperature + 2

print("\n(a) Original temperature readings:")
print(temperature)

print("\nCorrected temperature readings (+2°C):")
print(corrected_temperature)

# b. Convert Celsius to Fahrenheit
fahrenheit = (9/5) * corrected_temperature + 32

print("\n(b) Temperature in Fahrenheit:")
print(fahrenheit)
```

```

# c. Identify readings greater than 32°C
above_32 = corrected_temperature[corrected_temperature > 32]

print("\n(c) Readings greater than 32°C:")
print(above_32)

# d. Count readings greater than 32°C
count_above_32 = np.sum(corrected_temperature > 32)

print("\n(d) Number of readings greater than 32°C:")
print(count_above_32)

# e. Explanation
print("\n(e) Explanation:")
print("Vectorization allows NumPy to perform operations on the")
print("entire array without explicitly using a for loop.")
print("Boolean indexing allows us to select only the elements")
print("that satisfy a given condition.")
print("These techniques make code shorter, faster and more efficient.")

```

===== Q1. SENSOR TEMPERATURE READINGS =====

(a) Original temperature readings:

[25 28 31 35 38 27 33 40]

Corrected temperature readings (+2°C):

[27 30 33 37 40 29 35 42]

(b) Temperature in Fahrenheit:

[80.6 86. 91.4 98.6 104. 84.2 95. 107.6]

(c) Readings greater than 32°C:

[33 37 40 35 42]

(d) Number of readings greater than 32°C:

5

(e) Explanation:

Vectorization allows NumPy to perform operations on the entire array without explicitly using a for loop. Boolean indexing allows us to select only the elements

that satisfy a given condition.

These techniques make code shorter, faster and more efficient.

```
[4]: #Q2. DAILY STEPS MATRIX
# =====

print("\n\n" + "=" * 70)
print("Q2. DAILY STEPS MATRIX")
print("=" * 70)

steps = np.array([
    [5000, 6200, 7100],
    [8000, 7500, 9000],
    [4500, 5100, 4800],
    [9000, 8500, 9500]
])

print("\nSteps matrix:")
print(steps)

# a. Calculate total steps
total_steps = np.sum(steps)

print("\n(a) Total steps recorded:")
print(total_steps)

# b. Calculate mean
mean_steps = np.mean(steps)

print("\n(b) Mean number of steps:")
print(mean_steps)

# c. Maximum and minimum
maximum_steps = np.max(steps)
minimum_steps = np.min(steps)

print("\n(c) Maximum steps:")
print(maximum_steps)

print("Minimum steps:")
print(minimum_steps)

# d. Total steps for each day using axis=0
```

```

daily_total = np.sum(steps, axis=0)

print("\n(d) Total steps for each day (axis=0):")
print(daily_total)

# e. Total steps for each user using axis=1
user_total = np.sum(steps, axis=1)

print("\n(e) Total steps for each user (axis=1):")
print(user_total)

# f. Position containing maximum number of steps
flat_position = np.argmax(steps)

print("\n(f) Flattened index of maximum value:")
print(flat_position)

# Convert flattened position into row-column position
position = np.unravel_index(np.argmax(steps), steps.shape)

print("User/Day position of maximum value:")
print(position)

print("\nMaximum value is 9500.")
print("It belongs to User 4, Day 3.")

```

=====

Q2. DAILY STEPS MATRIX

=====

Steps matrix:

```

[[5000 6200 7100]
 [8000 7500 9000]
 [4500 5100 4800]
 [9000 8500 9500]]

```

(a) Total steps recorded:
84200

(b) Mean number of steps:
7016.666666666667

(c) Maximum steps:

9500

Minimum steps:

4500

(d) Total steps for each day (axis=0):

[26500 27300 30400]

(e) Total steps for each user (axis=1):

[18300 24500 14400 27000]

(f) Flattened index of maximum value:

11

User/Day position of maximum value:

(3, 2)

Maximum value is 9500.

It belongs to User 4, Day 3.

```
[5]: # Q3. NUMPY SLICING, COPYING, RESHAPING AND FLATTENING
# =====

print("\n\n" + "=" * 70)
print("Q3. NUMPY SLICING, COPYING, RESHAPING AND FLATTENING")
print("=" * 70)

# a. Create original array
original = np.array([1, 2, 3, 4, 5, 6])

print("\n(a) Original array:")
print(original)

# b. Create slice from index 1 to 4
subset = original[1:4]

print("\n(b) Subset from index 1 to 4:")
print(subset)

# c. Modify first element of subset
subset[0] = 999

print("\n(c) After modifying first element of subset to 999:")

print("Original array:")
print(original)
```

```

print("Subset:")
print(subset)

print("\nNote: The original array also changes because")
print("a normal NumPy slice creates a view of the original array.")

# d. Create another slice using copy()
copied_array = original[1:4].copy()

copied_array[0] = 500

print("\n(d) After creating a copy and modifying its first element to 500:")

print("Original array:")
print(original)

print("Copied array:")
print(copied_array)

print("\nNote: The original array does not change because")
print(".copy() creates an independent copy.")

# e. Create numbers 1 to 12 and reshape to 3x4
matrix = np.arange(1, 13).reshape(3, 4)

print("\n(e) 3 x 4 matrix:")
print(matrix)

# f. Indexing and slicing

# First row
first_row = matrix[0]

# Last row
last_row = matrix[-1]

# Second column
second_column = matrix[:, 1]

# Rows 1-2 and columns 2-3
selected_elements = matrix[0:2, 1:3]

print("\n(f) First row:")

```

```

print(first_row)

print("\nLast row:")
print(last_row)

print("\nSecond column:")
print(second_column)

print("\nElements from rows 1-2 and columns 2-3:")
print(selected_elements)

# g. Flatten using flatten() and ravel()
flatten_array = matrix.flatten()
ravel_array = matrix.ravel()

print("\n(g) Flatten using flatten():")
print(flatten_array)

print("\nFlatten using ravel():")
print(ravel_array)

# h. Modify ravel() array
ravel_array[0] = 100

print("\n(h) After modifying first element of ravel() to 100:")

print("Ravel array:")
print(ravel_array)

print("Original matrix:")
print(matrix)

print("\nObservation: ravel() usually returns a view,")
print("so changing it can affect the original array.")

# i. Modify flatten() array
flatten_array = matrix.flatten()

flatten_array[1] = 200

print("\n(i) After modifying second element of flatten() to 200:")

print("Flatten array:")
print(flatten_array)

```

```

print("Original matrix:")
print(matrix)

print("\nObservation: flatten() returns a copy,")
print("so changing it does not affect the original matrix.")

# j. Shape, ndim, size and dtype

print("\n(j) Properties of the original 3 x 4 matrix:")

print("Shape:", matrix.shape)
print("Number of dimensions (ndim):", matrix.ndim)
print("Number of elements (size):", matrix.size)
print("Data type (dtype):", matrix.dtype)

```

Q3. NUMPY SLICING, COPYING, RESHAPING AND FLATTENING

(a) Original array:

```
[1 2 3 4 5 6]
```

(b) Subset from index 1 to 4:

```
[2 3 4]
```

(c) After modifying first element of subset to 999:

Original array:

```
[ 1 999  3  4  5  6]
```

Subset:

```
[999  3  4]
```

Note: The original array also changes because a normal NumPy slice creates a view of the original array.

(d) After creating a copy and modifying its first element to 500:

Original array:

```
[ 1 999  3  4  5  6]
```

Copied array:

```
[500  3  4]
```

Note: The original array does not change because .copy() creates an independent copy.

(e) 3 x 4 matrix:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

(f) First row:

```
[1 2 3 4]
```

Last row:

```
[ 9 10 11 12]
```

Second column:

```
[ 2  6 10]
```

Elements from rows 1-2 and columns 2-3:

```
[[2 3]
 [6 7]]
```

(g) Flatten using `flatten()`:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

Flatten using `ravel()`:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

(h) After modifying first element of `ravel()` to 100:

Ravel array:

```
[100  2  3  4  5  6  7  8  9 10 11 12]
```

Original matrix:

```
[[100  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Observation: `ravel()` usually returns a view,
so changing it can affect the original array.

(i) After modifying second element of `flatten()` to 200:

Flatten array:

```
[100 200  3  4  5  6  7  8  9 10 11 12]
```

Original matrix:

```
[[100  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Observation: `flatten()` returns a copy,
so changing it does not affect the original matrix.

(j) Properties of the original 3 x 4 matrix:

Shape: (3, 4)

Number of dimensions (ndim): 2
Number of elements (size): 12
Data type (dtype): int64

```
[6]: # Q4. COGNITIVE-ASSISTIVE SYSTEM - ORDINARY LEAST SQUARES
# =====

print("\n\n" + "=" * 70)
print("Q4. COGNITIVE-ASSISTIVE SYSTEM - OLS")
print("=" * 70)

# Given X and y
X = np.array([
    [6, 70, 3],
    [5, 50, 6],
    [8, 80, 2],
    [4, 30, 8]
], dtype=float)

y = np.array([40, 65, 30, 85], dtype=float)

print("\nFeature matrix X:")
print(X)

print("\nTarget vector y:")
print(y)

# a. Shape and dimensions
print("\n(a) Shape of X:")
print(X.shape)

print("Dimensions of X:")
print(X.ndim)

# b. Transpose X
XT = X.T

print("\n(b) Transpose X (X.T):")
print(XT)

print("\nExplanation:")
print("Transpose changes rows into columns and columns into rows.")
print("The original X has shape (4, 3).")
```

```

print("The transpose has shape (3, 4).")

# c. Matrix product  $X.T @ X$ 
XT_X = X.T @ X

print("\n(c) Matrix product  $X.T @ X$ :")
print(XT_X)

# d. Matrix inverse
inverse_XT_X = np.linalg.inv(XT_X)

print("\n(d) Inverse of  $X.T @ X$ :")
print(inverse_XT_X)

# e. Ordinary Least Squares equation
#  $Beta = (X.T X)^{-1} X.T y$ 

beta = np.linalg.inv(X.T @ X) @ X.T @ y

print("\n(e) OLS coefficients (Beta):")
print(beta)

# f. Explain coefficients
print("\n(f) Interpretation of coefficients:")

print("Coefficient for Sleep hours:", beta[0])
print("Coefficient for Activity level:", beta[1])
print("Coefficient for Stress level:", beta[2])

print("\nThe coefficients represent the estimated change in")
print("assistance score associated with a one-unit change in")
print("each feature, while keeping the other features constant.")

# g. Predict assistance score for new user
new_user = np.array([5, 40, 7], dtype=float)

predicted_score = new_user @ beta

print("\n(g) New user:")
print(new_user)

print("\nPredicted assistance score:")

```

```

print(predicted_score)

# =====
# FINAL SUMMARY
# =====

print("\n\n" + "=" * 70)
print("ASSIGNMENT COMPLETED")
print("=" * 70)

print("\nQ1:")
print("Corrected temperatures:", corrected_temperature)
print("Fahrenheit:", fahrenheit)
print("Readings > 32°C:", above_32)
print("Count > 32°C:", count_above_32)

print("\nQ2:")
print("Total steps:", total_steps)
print("Mean steps:", mean_steps)
print("Maximum:", maximum_steps)
print("Minimum:", minimum_steps)
print("Daily totals:", daily_total)
print("User totals:", user_total)
print("Maximum position:", position)

print("\nQ3:")
print("Matrix:")
print(matrix)
print("Shape:", matrix.shape)
print("Dimensions:", matrix.ndim)
print("Size:", matrix.size)
print("Dtype:", matrix.dtype)

print("\nQ4:")
print("Beta coefficients:", beta)
print("New user:", new_user)
print("Predicted assistance score:", predicted_score)

print("\n" + "=" * 70)

```

```

=====
Q4. COGNITIVE-ASSISTIVE SYSTEM - OLS
=====

```

Feature matrix X:

```
[[ 6. 70.  3.]
 [ 5. 50.  6.]
 [ 8. 80.  2.]
 [ 4. 30.  8.]]
```

Target vector y:

```
[40. 65. 30. 85.]
```

(a) Shape of X:

```
(4, 3)
```

Dimensions of X:

```
2
```

(b) Transpose X (X.T):

```
[[ 6.  5.  8.  4.]
 [70. 50. 80. 30.]
 [ 3.  6.  2.  8.]]
```

Explanation:

Transpose changes rows into columns and columns into rows.

The original X has shape (4, 3).

The transpose has shape (3, 4).

(c) Matrix product X.T @ X:

```
[[ 141.  1430.   96.]
 [ 1430. 14700.  910.]
 [   96.   910.  113.]]
```

(d) Inverse of X.T @ X:

```
[[ 1.10521428 -0.09848746 -0.14581398]
 [-0.09848746  0.00891203  0.01190129]
 [-0.14581398  0.01190129  0.0368847  ]]
```

(e) OLS coefficients (Beta):

```
[-0.15921454  0.14083853 10.06302242]
```

(f) Interpretation of coefficients:

Coefficient for Sleep hours: -0.15921454159483517

Coefficient for Activity level: 0.14083852991905133

Coefficient for Stress level: 10.063022422714608

The coefficients represent the estimated change in assistance score associated with a one-unit change in each feature, while keeping the other features constant.

(g) New user:

```
[ 5. 40.  7.]
```

Predicted assistance score:
75.27862544779013

=====
ASSIGNMENT COMPLETED
=====

Q1:

Corrected temperatures: [27 30 33 37 40 29 35 42]
Fahrenheit: [80.6 86. 91.4 98.6 104. 84.2 95. 107.6]
Readings > 32°C: [33 37 40 35 42]
Count > 32°C: 5

Q2:

Total steps: 84200
Mean steps: 7016.666666666667
Maximum: 9500
Minimum: 4500
Daily totals: [26500 27300 30400]
User totals: [18300 24500 14400 27000]
Maximum position: (3, 2)

Q3:

Matrix:
[[100 2 3 4]
[5 6 7 8]
[9 10 11 12]]
Shape: (3, 4)
Dimensions: 2
Size: 12
Dtype: int64

Q4:

Beta coefficients: [-0.15921454 0.14083853 10.06302242]
New user: [5. 40. 7.]
Predicted assistance score: 75.27862544779013

[]: